

Highlights

A Stackelberg Inspection Game for Diagnosis–Repair in LLM-Based Empty Container Repositioning Optimization

Kangzhen Peng

- Diagnosis–repair is cast as a Stackelberg inspection game defined on a LangGraph state machine as the stage-game apparatus.
- The diagnosis agent is an inspector that commits, first-mover, to a causal-order (data $\triangleright$ model $\triangleright$ code) probing policy with executed refutation.
- Accused agents are inspectees with a genuine deflection incentive; the inspector’s commitment induces truthful attribution.
- A proposition links causal-order inspection to a strictly smaller viable root-cause set than single-judge diagnosis.

A Stackelberg Inspection Game for Diagnosis–Repair in LLM-Based Empty Container Repositioning Optimization

Kangzhen Peng^a

^aCollege of Transportation Engineering, Dalian Maritime University, No. 1 Linghai Road, Dalian, 116026, Liaoning, China

ARTICLE INFO

Keywords:

large language model
multi-agent system
Stackelberg game
finite-state machine
automated optimization modeling
empty container repositioning

ABSTRACT

We recast the error diagnosis and repair stage of an LLM multi-agent pipeline for Empty Container Repositioning (ECR) optimization as a Stackelberg *inspection game* defined on a state machine. A LangGraph finite-state machine orchestrates role-specialized agents (Data Engineer → Model Expert → Python Developer → Solver) and serves as the stage-game apparatus. When a solve fails, the diagnosis agent acts as an *inspector* (the Stackelberg leader) that commits, first and observably, to an inspection policy: a probing order following the pipeline’s causal dependency (data ▷ model ▷ code) together with *executed refutation* — verifying every response by a real re-solve rather than by cheap talk. The accused agents are *inspectees* (the followers): because a downstream layer is routinely implicated by an upstream defect it did not cause, each accused agent has a genuine private incentive to deflect blame and avoid wasted repair, so its interests are not aligned with truthful attribution. We show that the inspector’s committed causal-order, executed-refutation policy induces the truthful best response — a guilty layer complies and repairs, an innocent layer’s deflection is verified — replacing the baseline’s single-judge LLM accusation. We formalize the mechanism, state a proposition on causal-order inspection, and instantiate it on a *demand-uncertainty* sea–land ECR problem formulated as a two-stage stochastic linear program.

1. Introduction

Empty Container Repositioning (ECR) is a standing cost center in global liner shipping. Trade imbalance leaves empty equipment stranded at import hubs while export regions face shortages; planners must move empties across sea and inland networks under capacity, transit-time, and inventory limits (Poiani et al., 2024; Wang et al., 2025a). Under demand uncertainty the problem is naturally a multi-stage (often two-stage) stochastic program: first-stage repositioning commitments are made before scenario-dependent export demand realizes, and recourse actions (e.g. leasing or spill) absorb residual imbalance (Poiani et al., 2024; Cai et al., 2025). Building a correct model therefore requires scarce domain expertise: admissible sea–dry topologies, mode-specific costs, scenario trees or deterministic equivalents, and recourse design that remains relatively complete. Small misspecifications in data, mathematics, or solver code routinely yield infeasible, suboptimal, or operationally meaningless plans.

Traditional practice addresses ECR with expert-built mixed-integer or stochastic programs, meta-heuristics, and, more recently, multi-agent reinforcement learning for joint repositioning and fleet decisions (Poiani et al., 2024; Cai et al., 2025). These methods can be strong once a model exists, but they share a structural bottleneck: every new network, planning window, or uncertainty description must be *hand-modeled* by OR specialists. Reformulation is slow; knowledge is hard to transfer across carriers; and the gap between how operations are described in language and how they appear in an LP remains human-mediated. Learning-based controllers alleviate online decision load but still presume a correctly engineered state, action, and reward interface. In short, the limiting resource is often not the solver, but the modeling labor that feeds it.

Large language models (LLMs) change that interface. They parse operational narratives and structured tables, propose mathematical formulations, emit executable solver code, and iterate against tool feedback (Xiao et al., 2024; AhmadiTeshnizi et al., 2024a). These abilities align with ECR pain points: the input is already a mixture of natural language and instance data; the desired output is a compilable, solvable model whose quality can be checked by an industrial MIP solver and, when available, by a known optimum z^* . Multi-agent scaffolds (role-specialized data, modeling, and coding agents coordinated by a shared state) further match the layered work of OR practice, from

 pengkangzhen@dlmu.edu.cn (K. Peng)
ORCID(s):

data preparation through formulation to implementation (Wu et al., 2024; Xiao et al., 2024; AhmadiTeshnizi et al., 2024a). Recent maritime applications already use LLM agents for stowage-related planning and sketch agent-based transportation modeling (Liu et al., 2025), indicating that the same automation path is opening for container logistics.

Automation, however, relocates rather than removes the reliability problem. In a natural-language optimization (NLO) pipeline, defects propagate along *data* $\triangleright$ *model* $\triangleright$ *code*: an upstream modeling error often surfaces as a downstream crash or wrong objective, so stack traces mark the symptom locus rather than the certified root cause. Without domain knowledge the pipeline cannot formulate ECR correctly in the first place; with knowledge, failed solves still demand *attribution and repair*. Existing agentic OR systems rely mainly on single-judge accusation, sequential backward repair, multi-agent debate, or verbal reflection (Du et al., 2023; Shinn et al., 2023; Liang et al., 2024). These protocols increase deliberation but treat diagnosis as (multi-call) classification with cheap-talk self-checks. They do not commit an observable inspection policy, do not verify accused replies by executed re-solve as a discipline, and do not account for the private incentive of an innocent downstream layer to deflect blame.

Thesis. We argue that diagnosis–repair under layered ECR modeling errors is a Stackelberg *inspection game* (Avenhaus et al., 2002; Maschler, 1966). A diagnosis agent acts as inspector (leader) and commits, first and observably, to a policy $\sigma = (\omega, \nu)$: a probing order ω aligned with causal dependence, and a verification rule ν of *executed refutation* (a real re-solve under a strict-success test against z^*). Accused data, model, and code agents are inspectees (followers) that choose to comply-and-repair or to deflect. Commitment gives the mechanism teeth: a guilty layer’s best response is to repair; an innocent layer’s deflection is upheld only if re-solve confirms it. We instantiate the game on a LangGraph state machine as stage-game apparatus and validate it on a demand-uncertainty sea–land ECR two-stage DEP (Section 3).

Contributions. (i) We cast diagnosis–repair in an LLM multi-agent NLO pipeline as a Stackelberg inspection game on a finite-state machine. (ii) We specify an inspector commitment to causal-order probing with executed refutation, inducing verifiable attribution rather than confidence-routed accusation. (iii) We state a proposition that causal-order inspection strictly reduces the viable root-cause set relative to single-judge play without dropping the true layer, and test it with commitment-order ablations and baselines on demand-uncertainty ECR. (iv) We separate progressive domain-knowledge injection (necessary for ECR solvability) from the game-theoretic diagnosis claim (necessary for auditable repair).

Section 2 reviews ECR research and the shift to agentic NLO. Section 3 gives the ECR problem description and mathematical formulation. Section 4 develops the inspection mechanism. Section 5 states the experimental protocol; results and the Conclusion follow the campaign.

2. Related Work

2.1. Empty container repositioning

ECR has long been studied as a network optimization problem driven by directional trade imbalance. Classical and contemporary OR formulations use deterministic or stochastic mathematical programs for repositioning, inventory, slot allocation, and related fleet decisions, often solved by exact methods or meta-heuristics (Wang et al., 2025a; Cai et al., 2025). Under uncertainty and operational black-box constraints, configurable partially observable models and two-stage learning procedures have been proposed for joint repositioning and fleet deployment (Poiani et al., 2024). These lines establish ECR as a hard, domain-rich benchmark with verifiable optima once a model is fixed. They do not automate the passage from an operational description to that model. Emerging LLM-agent work in maritime and transportation systems (e.g. text-to-stowage pipelines and conceptual LLM-agent transportation frameworks) (Liu et al., 2025) suggests that natural-language interfaces are becoming plausible for container logistics, but reliability of the modeling–coding–solving chain remains open. We therefore use demand-uncertainty ECR as a *validation bed* with an expert DEP optimum z^* , not as a claim of a new ECR theory.

2.2. From expert modeling to agentic natural-language optimization

The broader OR+LLM literature is shifting how optimization problems are produced and checked. Chain-of-Experts orchestrates role-specialized agents with forward construction and backward reflection for complex OR tasks (Xiao et al., 2024); OptiMUS modularizes formulation, coding, and evaluation against solver feedback at scale (AhmadiTeshnizi et al., 2024a,b); related expert-guided reasoning pipelines further target automated optimization modeling (Yang et al., 2025). Relative to classical “expert writes the math, solver optimizes,” this body of work exhibits

three coupled shifts. First, the *object of study* moves from solving a given mathematical program to turning unstructured descriptions and data into a solvable, correct model. Second, *verification* moves from textual resemblance or manual reading toward executable checks (compile, solve, and comparison to z^* when available), so success is judged by whether the artifact passes a solver rather than by whether the prose looks right. Third, once modeling is a multi-stage pipeline, *reliability* becomes first-class: errors cascade along data, model, and code; the failure surface is often downstream of the true root cause. Domain knowledge injection is necessary for problems such as ECR to be formulable at all; failure attribution and diagnosis–repair are necessary for the pipeline to recover when it does not. Our work sits in this third shift while retaining executable solver verification as the hard truth signal.

2.3. Reliability mechanisms in LLM multi-agent systems

Layered LLM multi-agent systems face a distinctive reliability problem: errors cascade across role boundaries, and the observable failure surface (crash, infeasibility, or wrong objective) is often downstream of the true root cause. In NLO pipelines this appears as data $\triangleright$ model $\triangleright$ code dependence—a modeling mistake may surface as a coding crash or a wrong solve. Domain knowledge and role scaffolding are a *prerequisite* for hard OR tasks to be formulable at all (Wu et al., 2024; Xiao et al., 2024; AhmadiTeshnizi et al., 2024a), but they do not by themselves certify which layer failed or compel a correct repair. A complementary line therefore treats verification as executable rather than textual: compile, solve, and check against solver feedback or a known optimum when available (AhmadiTeshnizi et al., 2024a,b). Two further mechanism families address post-failure recovery. Failure attribution asks which agent and which step caused the failure: Who&When shows that even strong LLM judges remain far from reliable step-level attribution (Zhang et al., 2025); subsequent work stresses full-trace observability (Chen et al., 2026), hierarchical or causal-graph attribution (Wang et al., 2026), and Causal Agent Replay that intervenes on trajectory steps and re-executes forward (Shah, 2026). Diagnosis–repair and multi-call protocols then try to act on that judgment: single-judge accusation with status-based priors, reverse-order sequential repair, multi-agent debate (Du et al., 2023; Liang et al., 2024), and Reflexion-style verbal memory (Shinn et al., 2023). These lines increase deliberation or observability, yet they typically stop at naming (or reflective re-prompting). Missing for layered OR pipelines is a *committed* inspection policy whose replies are settled by re-solve under a strict success rule—the incentive-compatible, executed-refutation mechanism we develop as a Stackelberg inspection game in Section 2.4.

2.4. Game-theoretic multi-agent LLM systems

Game-theoretic views of LLM multi-agent systems cover debate, Nash-style coordination, and Stackelberg resource governance (Yi et al., 2025; Wang, 2026; Du et al., 2023). Classical inspection games analyze an inspector who commits to a monitoring policy and an inspectee who best-responds (Avenhaus et al., 2002; Maschler, 1966). We use that template for layered NLO diagnosis–repair: the diagnosis agent is the inspector; data/model/code agents are inspectees; commitment is to causal-order probing plus executed refutation on the ECR blackboard. This differs from resource Stackelberg controllers (token or tool budgets) and from debate equilibria (message-passing agreement) in both players and payoffs. The empirical spine is whether committed causal order improves *verifiable* attribution relative to unordered commitment and to non-committed single-judge or multi-call baselines (Section 5).

3. Problem Description and Mathematical Formulation

3.1. Problem description

We validate the diagnosis–repair mechanism on a sea–land Empty Container Repositioning (ECR) problem under *export-demand uncertainty*. The application model is a two-stage stochastic linear program on a weekly planning window (deterministic equivalent / DEP), adapted from our companion formulation for demand-uncertainty ECR. Unlike the MAKO baseline’s deterministic shipper–consignee multi-commodity flow, the present benchmark retains only sea and dry ports, treats empty supply as exogenous, and places all randomness in scenario-dependent export demand. Agents in the pipeline must formulate and solve this DEP from a natural-language description plus structured instance data; the expert DEP optimum supplies the strict-success rule used by the diagnosis mechanism (Section 4.4).

The network is a directed multigraph on node set $\mathcal{N} = \mathcal{H} \sqcup \mathcal{L}$, where sea ports $\mathcal{H} = \mathcal{H}^{\text{hub}} \sqcup \mathcal{H}^{\text{spoke}}$ partition into global hubs and regional spokes, and $\mathcal{L}$ is the set of dry ports. Inland moves use modes $\kappa \in \mathcal{K}$ (road, rail). Rather than writing Big-M hinterland restrictions inside the LP, we *compile* topology into an admissible arc set

$$\mathcal{A} \subseteq \mathcal{N} \times \mathcal{N} \times \mathcal{K}, \quad (1)$$

built once from hinterland indicators λ_{ij} : include sea–dry arcs (i, j, κ) and (j, i, κ) iff $\lambda_{ij} = 1$, and dry–dry arcs (j_1, j_2, κ) iff j_1 and j_2 share a common hinterland sea port. Each arc $a = (i, j, \kappa) \in \mathcal{A}$ carries transit time τ_a , unit repositioning cost c_a^{repo} , and capacity F_a^{cap} . Flow variables exist only on $\mathcal{A}$; inadmissible hinterland pairs simply never appear.

Time is discrete by week. Empty containers held at sea or dry ports serve export demand. The empty supply $\xi_{n,t}$ at node n in week t is an *exogenous deterministic* parameter (no laden-to-empty conversion dynamics). The core uncertainty is the scenario-dependent export demand $\eta_{n,t}^\omega$ for $\omega \in \Omega$ with probabilities π_ω . Leasing is unbounded (at a premium); a heavily penalized spill variable ensures relatively complete recourse under storage caps.

A full operational system may wrap successive windows in a rolling horizon. For the agentic modeling task we fix a single window $\mathcal{T} = \{1, \dots, H\}$ (smoke instance: $H=4, |\Omega|=5$) and ask the pipeline to build the extensive-form DEP of the two-stage program below. All decisions are continuous nonnegative TEUs; at weekly scale the integrality gap is negligible.

3.2. Mathematical formulation

3.2.1. Sets, parameters, and variables

Sets. $\mathcal{H}^{\text{hub}}, \mathcal{H}^{\text{spoke}}, \mathcal{L}; \mathcal{N}^{\text{inv}} = \mathcal{H} \cup \mathcal{L}$; modes $\mathcal{K}$; admissible arcs $\mathcal{A}$ as in (1); scenarios Ω ; window $\mathcal{T}$. For each node $n \in \mathcal{N}$ define the forward and backward stars

$$\mathcal{A}_n^+ = \{(n, j, \kappa) \in \mathcal{A}\}, \quad \mathcal{A}_n^- = \{(i, n, \kappa) \in \mathcal{A}\}. \quad (2)$$

Flow variables are indexed on arcs $(i, j, \kappa) \in \mathcal{A}$; we write $x_{ijk,t}^\omega$ interchangeably with $x_{a,t}^\omega$ for $a = (i, j, \kappa)$.

Deterministic parameters. Exogenous supply $\xi_{n,t}$; spoke top-up $E_{i,t}$; initial inventory $I_{n,0}$; committed inland arrivals $\bar{Q}_{n,t}^{\text{arr}}$ (zero on the first-window smoke instance); hub vessel calls $V_{i,t} \in \{0, 1\}$ and per-call bounds $B_i^{\text{in}}, B_i^{\text{out}}$; effective external export floor $\bar{D}_{i,t}^{\text{ext}} = \min\{D_{i,t}^{\text{ext}}, B_i^{\text{out}}\}$; per-arc $(\tau_{ijk}, c_{ijk}^{\text{repo}}, F_{ijk}^{\text{cap}})$ for $(i, j, \kappa) \in \mathcal{A}$; storage caps U_n^{cap} ; sea costs $c^{\text{sea,in}}, c^{\text{sea,out}}$; repositioning / holding / leasing / spill costs $c_{ijk}^{\text{repo}}, c_n^{\text{hold}}, c_n^{\text{lease}}, c^{\text{spill}}$, with $c^{\text{spill}} \gg \max\{c_n^{\text{lease}}, c_n^{\text{hold}}, c_{ijk}^{\text{repo}}\}$.

Random parameters. Demand $\eta_{n,t}^\omega$ and probabilities π_ω ($\sum_\omega \pi_\omega = 1$). Scenarios are drawn by truncated-normal additive shocks about nominal means $\bar{\eta}_{n,t}$.

First-stage decisions (here-and-now). $y_{i,t}^{\text{in}}, y_{i,t}^{\text{out}} \in \mathbb{R}_+$ for $i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}$: hub sea arrivals (arrival-week accounting) and departures, chosen *before* demand is revealed.

Second-stage decisions (wait-and-see). For each $\omega \in \Omega$: arc flows $x_{ijk,t}^\omega$ ($(i, j, \kappa) \in \mathcal{A}$), inventories $I_{n,t}^\omega$, leases $r_{n,t}^\omega$, and spill $s_{n,t}^\omega$, all in $\mathbb{R}_+$.

3.2.2. Mathematical model

The inland submodel is a *node–arc* network formulation on the admissible digraph $(\mathcal{N}, \mathcal{A})$: topology is encoded by the stars (2), and conservation at each node is written with incidence sums over $\mathcal{A}_n^\pm$.

Stage 1 master problem [D1].

$$\min_{\mathbf{y}} \sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{H}^{\text{hub}}} (c^{\text{sea,in}} y_{i,t}^{\text{in}} + c^{\text{sea,out}} y_{i,t}^{\text{out}}) + \sum_{\omega \in \Omega} \pi_\omega \Theta^D(\mathbf{y}, \omega) \quad (3)$$

$$\text{s.t. } y_{i,t}^{\text{in}} \leq B_i^{\text{in}} V_{i,t}, \quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \quad (4)$$

$$y_{i,t}^{\text{out}} \leq B_i^{\text{out}} V_{i,t}, \quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \quad (5)$$

$$y_{i,t}^{\text{out}} \geq \bar{D}_{i,t}^{\text{ext}} V_{i,t}, \quad \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \quad (6)$$

$$y_{i,t}^{\text{in}}, y_{i,t}^{\text{out}} \geq 0. \quad (7)$$

Stage 2 recourse [D2]. Given $\mathbf{y}$ and realized η^ω , write the node–arc inflow and outflow through the stars (2) as

$$\begin{aligned} \text{In}_{n,t}^\omega &= \sum_{(i,n,\kappa) \in \mathcal{A}_n^-} x_{in\kappa,t-\tau_{in\kappa}}^\omega, \\ \text{Out}_{n,t}^\omega &= \sum_{(n,j,\kappa) \in \mathcal{A}_n^+} x_{njk,t}^\omega, \end{aligned} \quad (8)$$

with the convention $x_{ijk,s}^\omega := 0$ if $s \notin \mathcal{T}$ (lagged arrivals outside the window enter via $\bar{Q}_{n,t}^{\text{arr}}$). The recourse problem is

$$\Theta^D(\mathbf{y}, \omega) = \min_{\mathbf{x}, \mathbf{I}, \mathbf{r}, \mathbf{s}} \left(C^{\text{repo}}(\omega) + C^{\text{hold}}(\omega) + C^{\text{lease}}(\omega) + C^{\text{spill}}(\omega) \right), \quad (9)$$

where the four activity cost blocks (repositioning / holding / leasing / spill) are

$$C^{\text{repo}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{(i,j,\kappa) \in \mathcal{A}} c_{ijk}^{\text{repo}} x_{ijk,t}^\omega, \quad (10)$$

$$C^{\text{hold}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}^{\text{inv}}} c_n^{\text{hold}} I_{n,t}^\omega, \quad (11)$$

$$C^{\text{lease}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}^{\text{inv}}} c_n^{\text{lease}} r_{n,t}^\omega, \quad (12)$$

$$C^{\text{spill}}(\omega) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}^{\text{inv}}} c_n^{\text{spill}} s_{n,t}^\omega. \quad (13)$$

subject to

$$\begin{aligned} I_{i,t}^\omega &= I_{i,t-1}^\omega + \xi_{i,t} + r_{i,t}^\omega + y_{i,t}^{\text{in}} - y_{i,t}^{\text{out}} + \bar{Q}_{i,t}^{\text{arr}} \\ &\quad + \text{In}_{i,t}^\omega - \text{Out}_{i,t}^\omega - \eta_{i,t}^\omega - s_{i,t}^\omega, \\ \forall i \in \mathcal{H}^{\text{hub}}, t \in \mathcal{T}, \end{aligned} \quad (14)$$

$$\begin{aligned} I_{i,t}^\omega &= I_{i,t-1}^\omega + \xi_{i,t} + r_{i,t}^\omega + E_{i,t} + \bar{Q}_{i,t}^{\text{arr}} \\ &\quad + \text{In}_{i,t}^\omega - \text{Out}_{i,t}^\omega - \eta_{i,t}^\omega - s_{i,t}^\omega, \\ \forall i \in \mathcal{H}^{\text{spoke}}, t \in \mathcal{T}, \end{aligned} \quad (15)$$

$$\begin{aligned} I_{j,t}^\omega &= I_{j,t-1}^\omega + \xi_{j,t} + r_{j,t}^\omega + \bar{Q}_{j,t}^{\text{arr}} \\ &\quad + \text{In}_{j,t}^\omega - \text{Out}_{j,t}^\omega - \eta_{j,t}^\omega - s_{j,t}^\omega, \\ \forall j \in \mathcal{L}, t \in \mathcal{T}, \end{aligned} \quad (16)$$

$$0 \leq x_{ijk,t}^\omega \leq F_{ijk}^{\text{cap}}, \quad \forall (i,j,\kappa) \in \mathcal{A}, t \in \mathcal{T}, \quad (17)$$

$$0 \leq I_{n,t}^\omega \leq U_n^{\text{cap}}, \quad \forall n \in \mathcal{N}^{\text{inv}}, t \in \mathcal{T}, \quad (18)$$

$$r_{n,t}^\omega, s_{n,t}^\omega \geq 0, \quad \forall n \in \mathcal{N}^{\text{inv}}, t \in \mathcal{T}, \quad (19)$$

with $I_{n,0}$ fixed as the window initial inventory. Equations (14)–(16) are node–arc balances; the star sums (8) enforce hinterland feasibility through $\mathcal{A}_n^\pm \subseteq \mathcal{A}$. The remaining inequalities are arc capacities (17) and yard caps (18). Lease variables are unbounded above; spill s with large c^{spill} provides relatively complete recourse.

3.2.3. Success criterion for the agentic task

The agentic task is to emit a Gurobi implementation of the extensive-form DEP equivalent to (3)–(19). An expert DEP solve on the frozen smoke instance yields a known optimum z^* (approximately 1.390×10^6 for $H=4$, $|\Omega|=5$, seed 42). Critically, a solver report of OPTIMAL is *not* by itself a certificate that the emitted model is the intended DEP: an incomplete or misspecified formulation can still be feasible and yield a spurious optimum $\hat{z} \neq z^*$. Table 1 therefore distinguishes four terminal outcomes by crossing the solver status with model correctness relative to z^* .

Here *model correct* means the relative gap $|\hat{z} - z^*|/|z^*|$ does not exceed 10^{-2} (one percent; the experimental default). A pipeline run is counted as a *strict success* (SSR) iff the outcome is *Practical Optimal*: the solver reports OPTIMAL and the gap test against z^* passes. OPTIMAL alone with a failed gap test is classified as *Spurious Optimal* and does not count toward SSR. Injected root-cause labels for attribution experiments are defined on the same layered pipeline (data $\triangleright$ model $\triangleright$ code) and are independent of the OR model above.

Table 1

Outcome categories mapping solver status to model correctness.

Outcome	Solver OPTIMAL?	Model correct?
Practical Optimal	✓	✓
Spurious Optimal	✓	×
Model Infeasibility	×	×
Runtime Error	(not reached)	(not reached)

4. Proposed Method

This section develops the proposed framework bottom-up. We define the agents and their interfaces (Section 4.1), compose them into a state-machine architecture (Section 4.2), and specify their communication through a shared blackboard (Section 4.3). The final two subsections present the core contribution: a Stackelberg *inspection game* for diagnosis–repair defined *on* the state machine (Section 4.4) and its algorithmic realization (Section 4.5). Throughout, the framework targets *layered-error* natural-language optimization — OR pipelines with a data $\rightarrow$ model $\rightarrow$ code dependency in which errors propagate strictly downstream and contaminate downstream signals; we do not claim gains for general LLM reasoning.

4.1. Agent Definitions

We retain the baseline’s role-specialized agents. Let $\mathcal{A} = \{a_1, \dots, a_5\}$, with $a_1 = \text{DataEngineer}$, $a_2 = \text{ModelExpert}$, $a_3 = \text{PythonDeveloper}$, $a_4 = \text{SolverExecutor}$, $a_5 = \text{DiagnosisAgent}$. Each agent is characterized by a tuple

$$a_i = (i, \mathcal{R}_i, \mathcal{T}_i, \mathcal{B}_i, \mathcal{H}_i), \quad (20)$$

where i is a unique identifier, $\mathcal{R}_i$ the functional role, $\mathcal{T}_i$ the prompt template encoding role-specific instructions and output-format constraints, $\mathcal{B}_i$ an optional knowledge base (active only for a_2 , whose modeling decisions are domain-specialized), and $\mathcal{H}_i$ the hyper-parameter configuration (temperature, top_p, max_tokens).

The five roles decompose the modeling task along the causal order $\text{data} \triangleright \text{model} \triangleright \text{code}$. DataEngineer (a_1) parses the problem description and raw instance into structured parameters with symbol anchoring for downstream consistency; ModelExpert (a_2) designs the optimization model (decision variables, objective, constraints); PythonDeveloper (a_3) translates the formulation into executable solver code; SolverExecutor (a_4) runs the code in a sandbox, validates the result against the strict-success rule, and reports status; DiagnosisAgent (a_5) analyzes any failure and drives repair.

Each correctable agent exposes two maps: a *forward* map a_i^+ that produces its nominal output, and a *backward* (repair) map a_i^- that, given an error signal, either accepts responsibility and emits a corrected output or rebuts. Agent a_i receives an input assembled from prior outputs,

$$x^{(i)} \leftarrow f_i(y^{(0)}, y^{(1)}, \dots, y^{(i-1)}), \quad y^{(i)} \leftarrow a_i^+(x^{(i)}), \quad (21)$$

where f_i selects the necessary dependencies. The backward map is invoked only on the agent implicated by diagnosis (Section 4.4).

4.2. Multi-Agent Architecture and State-Machine Orchestration

The agents of Section 4.1 are composed into an extended finite-state machine rather than a free-form graph. The pipeline is compiled as

$$M = (\mathcal{Q}, q_0, \Delta, S, K), \quad (22)$$

where $\mathcal{Q}$ is a finite set of control states (the graph nodes), q_0 the unique entry state, Δ the transition map, S the shared blackboard (Section 4.3), and K the transition budget. The state set is

$$\mathcal{Q} = \{q_{\text{DE}}, q_{\text{ME}}, q_{\text{KL}}, q_{\text{PD}}, q_{\text{SE}}, q_{\text{DG}}\} \cup \{q_{\text{DE}}^-, q_{\text{ME}}^-, q_{\text{PD}}^-\}, \quad (23)$$

comprising six forward states — one per agent in $\{a_1, \dots, a_5\}$ plus a dedicated KNOWLEDGELOADER state — and three backward (repair) states, one per correctable agent. The entry state is $q_0 = q_{\text{DE}}$ and the lone terminal is END. Figure 1 depicts M .

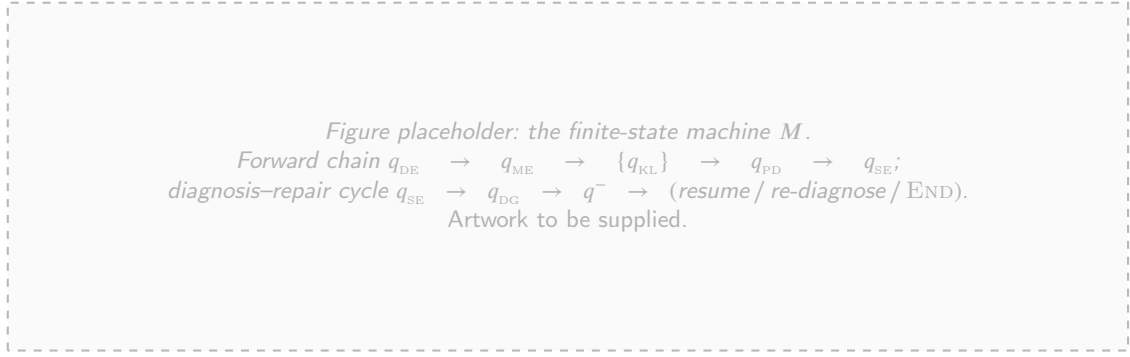


Figure 1: The extended finite-state machine $M = (Q, q_0, \Delta, S, K)$. Solid arrows: the forward chain (including an optional ModelExpert knowledge-load branch). Dashed arrows: the diagnosis–repair cycle, whose return edge is determined by the Stackelberg verdict (Section 4.4).

Transitions are pure functions of the blackboard. The transition map Δ is realized by five routing functions, each a pure map $S \rightarrow Q$:

- δ_{ME} : after ModelExpert, branch to KNOWLEDGELOADER iff it requests not-yet-loaded modules, else to PythonDeveloper;
- δ_{KL} : after KnowledgeLoader, return to ModelExpert iff new knowledge was added, else proceed to PythonDeveloper;
- δ_{SE} : after SolverExecutor, terminate on success or budget exhaustion, otherwise route to DiagnosisAgent;
- δ_{DG} : after DiagnosisAgent, route to the backward state of the implicated agent (the diagnosis verdict);
- δ^- : after a backward step, either resume the pipeline downstream from the repaired agent, re-enter diagnosis, or terminate — depending on whether the repair was accepted and which agent was repaired.

The retry budget K (`max_retries`) bounds the number of diagnosis–repair cycles, i.e., it is the FSM’s transition budget over the diagnostic sub-graph.

Domain knowledge is progressive scaffolding, not the contribution. ModelExpert does not receive a dump of every domain file, nor a RAG retriever. It first sees a *catalog* (module name + short description); it may then request modules by name; KNOWLEDGELOADER injects only those full texts and returns control for refinement. This on-demand path is required for correct TSLP modeling but is orthogonal to the Stackelberg inspection game: both features plug into the same FSM without owning each other’s logic.

The FSM is the stage-game apparatus, not scaffolding. The state machine is the structural substrate of the contribution because it is the object on which the repair game is defined. Its states are stage-game states, and — once the repair game is in play — the transitions δ_{DG} and δ^- become functions of the players’ strategy profile (the inspector’s committed inspection policy and the inspectee’s verified response; Section 4.4) rather than of a fixed heuristic. Treating M as the stage-game apparatus is what lets the commitment order carry *causal* meaning and what supports the proposition in Section 4.4; without this formalization, the FSM would be indistinguishable from generic graph orchestration.

4.3. Inter-Agent Communication via a Shared Blackboard

Agents never exchange messages pairwise. All communication is mediated by a single typed blackboard S (realized as the `AgentState` schema): every node reads its inputs from S and writes its outputs back to S , and the transition functions Δ route solely on S ’s contents. This *shared-state, no-direct-links* design has three consequences.

Control and payload on one schema, separated by reader. S carries both *control state* and *data payload*. Control fields — retry count, the currently implicated agent, a resolved flag, the diagnosis mode, the round counter, the retry budget — drive Δ and are the only fields a routing function inspects. Data fields — each agent’s output, the assembled error evidence, execution metrics, the loaded knowledge modules, and the ground-truth objective — carry the pipeline’s content and are read by the agent that fires next. Colocating these concerns on one schema while separating their readers is what makes the FSM’s transitions both data-aware and cheap to reason about.

A consistent, fixed data view. A deterministic preprocessor runs once, before any LLM agent fires, and writes a canonical view of the raw instance into S . Every downstream agent therefore reasons over an identical data basis, eliminating a class of data-parsing races that would otherwise masquerade as modeling or coding errors and confuse the diagnosis mechanism.

Append-by-field history. Communication is append-by-field rather than overwrite: a backward step writes a corrected output together with a resolved flag without erasing the prior forward outputs. The diagnosis mechanism can therefore inspect the full input–output history $\{(x^{(i)}, y^{(i)})\}$ when attributing a failure. This history is the evidence base $\mathcal{E}$ on which the Stackelberg game of Section 4.4 is defined, and its completeness is what makes blame attributable rather than heuristic.

4.4. The Stackelberg Diagnosis–Repair Game as an Inspection Game

We cast the diagnosis–repair stage as a Stackelberg game on the FSM multi-agent pipeline. The `DiagnosisAgent` moves first and commits — observably — to an inspection policy; the accused correctable agents then best-respond. Leader commitment plus follower response matches the pipeline’s control structure: the orchestrator already routes through diagnosis before any repair is applied, so the natural extensive form is sequential rather than simultaneous classification.

4.4.1. Causal layering of defects and the deflection incentive

Let the correctable core of the pipeline be the chain $a_1 \triangleright a_2 \triangleright a_3$ (`DataEngineer` $\triangleright$ `ModelExpert` $\triangleright$ `PythonDeveloper`), i.e. *data* $\triangleright$ *model* $\triangleright$ *code*, where $\triangleright$ denotes “is a causal prerequisite of.” We assume *strict causal layering of defects*: a defect introduced at layer $\ell \in \{1, 2, 3\}$ may manifest at any layer $j \geq \ell$, but a defect at layer ℓ cannot be the root cause of a symptom that requires only layers $< \ell$ to be well-formed. Consequently, the observation of a downstream failure (e.g. an infeasible solve, a wrong objective, or a traceback into solver-facing code) is consistent with several root-cause layers, and localizing the true root cause ℓ^* from the failure evidence $\mathcal{E}$ alone is fundamentally ambiguous: $\mathcal{E}$ records the *symptom locus* (often a_3), not a certified identity of ℓ^* . In particular, one cannot read the root-cause layer off the `SolverExecutor`’s failure surface alone: a Python traceback or crash locus typically marks where the pipeline *broke*, most often in generated code, rather than where the defect was *introduced*. The failure surface therefore motivates an *inspection* protocol rather than a one-shot classification of the traceback locus.

Causal layering of defects does not merely make attribution ambiguous; it gives the accused agents *misaligned private incentives*, which is what turns diagnosis into a game rather than a classification. Consider an agent at layer ℓ that a failure has implicated. Two facts hold simultaneously: (i) repair is costly — a backward step re-generates the agent’s output, consumes tokens, and risks injecting a fresh defect; and (ii) if $\ell \neq \ell^*$ (the layer is innocent, merely downstream of the true cause), then *no* local repair at ℓ can make the solve pass, because by strict causal layering of defects the true defect lies upstream. An innocent implicated agent therefore strictly prefers to *deflect* — to rebut the accusation and point blame upstream — rather than to accept a repair that wastes effort and cannot succeed. Symmetrically, even a guilty layer, if it could avoid the repair cost by deflecting, would be tempted to do so. The accused agents are thus not neutral truth-seekers whose objective coincides with the diagnostician’s; they are self-interested inspectees. Resolving this requires an inspector that *commits* to verification, which is what Section 4.4.2 constructs.

This incentive structure is exactly that of an inspectee facing an inspector. We accordingly specialize the stage game to a *Stackelberg inspection game* (Avenhaus et al., 2002; Maschler, 1966), a canonical class of leader–follower games in which an inspector commits to a monitoring policy and an inspectee best-responds. The diagnosis agent is the inspector (leader); the accused correctable agents are inspectees (followers); and the inspector’s first-mover *commitment* to a causal-order, executed-refutation policy is what induces truthful attribution as the inspectees’ best response. Table 2 maps the inspection-game elements to the pipeline and contrasts them with non-committed, single-judge protocols.

Table 2

Mapping of Stackelberg inspection-game elements to the diagnosis–repair stage. Right column: what non-committed / single-judge protocols provide instead.

Inspection-game element	Concretization in our mechanism	Non-committed / single-judge protocols
Inspector / leader (moves first, commits)	DiagnosisAgent commits an inspection policy σ : probe order + executed refutation	One LLM call picks a suspect
Inspectees / followers (best-respond)	Accused agents a_1, a_2, a_3 : comply-and-repair or deflect-upstream	Accused agent self-checks in isolation
Committed policy σ	Causal-order probing + verify by real re-solve (no cheap talk)	Fixed heuristic prior + single confidence
Conflict of interest	Innocent layer deflects to avoid wasted repair / wrongful blame	(assumed absent)
Inspector payoff	Correct root-cause localization at fewest rounds/tokens	(undefined)
Inspectee payoff	Avoid wrongful blame and wasted repair; fix only if truly at fault	A confidence score
Induced outcome	Guilty layer complies; innocent deflection is verified	Confidence-routed re-accusation
Stage-game apparatus	The LangGraph FSM (states = stage-game states)	(treated as plumbing)

4.4.2. Game formulation

A diagnosis–repair stage game G is instantiated whenever Δ routes from q_{SE} to q_{DG} (the solver has failed and the budget is not exhausted). The game is played on the current blackboard $S_t \in \mathcal{S}$ and has the following structure.

Players. An *inspector* I (the leader) and a set of *inspectees* (the followers). The inspector is the DiagnosisAgent a_5 , operating as an independent examiner. The inspectees are the correctable agents a_1, a_2, a_3 ; in a given probe the active inspectee is the agent $a_{\hat{\ell}}$ at the currently probed layer $\hat{\ell}$.

Timing. Play is Stackelberg: the inspector first commits to an observable inspection policy; the active inspectee then responds; the inspector finally settles the response by verification. The probing order ω follows the causal order $a_1 \triangleright a_2 \triangleright a_3$: the inspector probes the seeded suspect layer first and, when a layer is cleared, descends to the next causal layer. Aligning commitment order with causal order makes the first move causally meaningful (an upstream probe is settled by re-solve before the inspector descends) rather than a coin flip among observationally equivalent layers.

Strategies. The inspector’s strategy is an *inspection policy* $\sigma = (\omega, \nu)$ fixed *before* any inspectee responds: (i) a probing order ω over the layers, and (ii) a verification rule ν that resolves every response by *executed refutation* rather than by accepting the inspectee’s word. When probed at layer ℓ , the inspectee chooses $r \in \{\text{COMPLY}, \text{DEFLECT}\}$. To COMPLY is to accept responsibility and emit a concrete repair $h \in \mathcal{H}_{\ell}$; to DEFLECT is to rebut the accusation and assert that the root cause lies upstream. Given evidence $\mathcal{E}$ from the blackboard — stack trace, solver log, implicated code, model definition, data structures, and pipeline input–output pairs $\{(x^{(i)}, y^{(i)})\}$ — the active inspectee emits

$$(r, h) \leftarrow a_{\hat{\ell}}^{-}(x^{(\hat{\ell})}, y^{(\hat{\ell})}, \mathcal{E}), \quad r \in \{\text{COMPLY}, \text{DEFLECT}\}, \quad (24)$$

where, if $r = \text{COMPLY}$, h carries a justified repair, and if $r = \text{DEFLECT}$, h carries an upstream rebuttal.

Verification. Under its committed rule ν , the inspector does not take the response on trust. It selects and *executes* the cheapest decisive test,

$$\tau^* \leftarrow \arg \min_{\tau \in \mathcal{T}(r, h)} \text{cost}(\tau) \quad \text{s.t. } \tau \text{ is decisive}, \quad (25)$$

— a re-solve after applying h when $r = \text{COMPLY}$, or a targeted re-solve isolating the named upstream layer when $r = \text{DEFLECT}$ — and issues a verdict

$$v = (\beta, \hat{\ell}^*) = I(r, h, \tau^*, S_t), \quad (26)$$

where $\beta \in \{0, 1\}$ records whether the probed layer is *cleared* ($\beta=1$: a compliance whose re-solve still fails, or a deflection the inspector upholds) or *confirmed* ($\beta=0$: a compliance whose re-solve passes the strict-success rule), and $\hat{\ell}^*$ is the updated root-cause estimate. Executing the test supplies a grounded signal: neither a false confession nor a dishonest deflection survives. The FSM transition δ^- routes on this verdict: if $\beta = 0$, the repair in h is committed and Δ resumes downstream of $a_{\hat{\ell}}$; if $\beta = 1$, Δ re-enters diagnosis and the inspector probes the next causal layer, until confirmation or budget exhaustion (K).

Payoffs. Because LLMs admit no stable, observable utility function, we do *not* claim that an LLM “feels” a payoff. Instead we define numeric payoffs that the orchestrator optimizes and treat each LLM call as a *black-box best-responder*, analyzing the resulting trajectory. The inspector’s payoff rewards correct localization at low cost,

$$u_I(\sigma) = \mathbf{1}[\hat{\ell}^* = \ell^*] - \lambda_I \text{cost}(\sigma), \quad (27)$$

while the inspectee at layer ℓ , responding r , has the misaligned payoff

$$u_{\ell}(r; \sigma) = b \cdot \mathbf{1}[r=\text{COMPLY} \wedge \text{re-solve passes}] - c_{\text{rep}} \mathbf{1}[r=\text{COMPLY}] - c_{\text{blame}} \mathbf{1}[\text{held responsible} \wedge \ell \neq \ell^*], \quad (28)$$

where $b > 0$ is the reward for actually resolving the failure, $c_{\text{rep}} > 0$ the cost/risk of a repair, $c_{\text{blame}} > 0$ the penalty for carrying blame that does not fix the solve, $\text{cost}(\cdot)$ is token/round cost, and $\lambda_I \geq 0$ trades accuracy against budget. Eq. (28) makes the conflict explicit: the inspectee is rewarded only when it is *both* at fault *and* repairs, and is otherwise penalized for complying, so its interests diverge from the inspector’s whenever $\ell \neq \ell^*$. The ground-truth label ℓ^* enters only the *evaluation* of payoffs, never any agent’s prompt, so the game remains well-defined at inference time; equilibria are properties of the orchestrator-defined game, while each LLM is a sampled best-responder within it.

4.4.3. Induced truthful attribution

The value of the inspector’s first move is that its committed executed-refutation rule v reshapes the inspectee’s best response — the classical inspection-game deterrence effect. Consider the two cases. (i) *Innocent inspectee* ($\ell \neq \ell^*$): by strict causal layering of defects a local repair cannot make the solve pass, so COMPLY yields $-c_{\text{rep}}$ (no reward b , and it will still be seen not to fix the solve), whereas a truthful DEFLECT avoids the repair cost and is upheld by the inspector’s targeted re-solve. Deflection strictly dominates — and, crucially, only a *truthful* deflection survives v , since a dishonest one is refuted by re-solve. (ii) *Guilty inspectee* ($\ell = \ell^*$): deflecting cannot succeed, because every upstream probe the inspector runs under causal order passes verification and the probe returns to ℓ^* having only consumed rounds; complying and repairing yields $b - c_{\text{rep}}$. Whenever $b > c_{\text{rep}}$ (a resolved solve is worth more than the repair), COMPLY is the guilty layer’s best response. Hence the committed policy makes *truthful attribution* — the guilty complies, the innocent deflects honestly — the induced best-response profile. It is the inspector’s *commitment*, not any assumed benevolence of the agents, that aligns individually optimal responses with correct root-cause localization. Without commitment, deflection is cheap talk and truthful attribution is not incentive-compatible.

4.4.4. Proposition

Proposition 1 (Causal-order inspection lowers mis-attribution)

Under strict causal layering of defects and an inspector committed to a causal-order, executed-refutation policy, each probe is *decisive*: it either confirms the probed layer (a compliance whose re-solve passes) or clears it (a false confession refuted, or a truthful deflection upheld). Consequently, causal-order inspection strictly reduces the per-round set of viable root-cause hypotheses relative to simultaneous (single-judge) play, and never eliminates the true layer. Formally, let V_{sim} and V_{ins} be the sets of root-cause layers that remain consistent with the failure observation after one round of, respectively, simultaneous diagnosis and causal-order inspection; then $V_{\text{ins}} \subseteq V_{\text{sim}}$ with $\ell^* \in V_{\text{ins}}$, and the inclusion is strict whenever a probed layer $\ell < \ell^*$ is cleared.

Sketch. Under simultaneous play the diagnostician selects a hypothesis without observing any committed, verified response, so every layer consistent with the downstream observation is a candidate; the viable set is the full

observationally-equivalent upstream cone. Under committed inspection the inspectee at the probed layer ℓ must respond and the response is settled by executed refutation. If $\ell < \ell^*$, no local repair can pass the re-solve (strict causal layering of defects), so a compliance is refuted and a deflection upheld — either way ℓ is cleared and removed from the viable set. If $\ell = \ell^*$, the incentive analysis of Section 4.4.3 makes compliance the best response and the re-solve passes, so the true layer is confirmed and never wrongly eliminated. Causal ordering guarantees layers are cleared from the top of the upstream cone downward, so each cleared probe strictly shrinks V while simultaneous play can only guess within it. A formal bound on the mis-attribution rate and its empirical validation — centered on the commitment-order ablation — are deferred to the experimental section. $\square$

4.5. The Diagnosis–Repair Algorithm

Algorithm 1 unifies the architecture and the repair game. The FSM drives the forward pass; on failure it enters the inspection stage, where the inspector commits to a causal-order, executed-refutation policy and probes layers in causal order — each probe eliciting an inspectee response and settling it by re-solve — until a layer is confirmed or the budget K is spent. The transition map Δ is exactly the five routing functions of Section 4.2, with the return edge out of the backward state set by the executed verdict v .

Algorithm 1 FSM-Stackelberg diagnosis–repair (inspection game)

Require: Problem $\mathcal{P}$; agents $\{a_1, a_2, a_3, a_4\}$; inspector role a_5 ; FSM $M = (Q, q_0, \Delta, S, K)$; causal order $a_1 \triangleright a_2 \triangleright a_3$

Ensure: Result $Y \in \{\text{success}, \text{failure}\}$

```

1:  $S \leftarrow \text{PREPROCESS}(\mathcal{P})$ ;  $k \leftarrow 0$ ;  $Y \leftarrow \perp$ 
2: while  $Y = \perp$  do  $\triangleright$  drive the FSM
3:   // Forward chain:  $q_{\text{DE}} \rightarrow q_{\text{ME}} \rightarrow \{q_{\text{KL}}\} \rightarrow q_{\text{PD}} \rightarrow q_{\text{SE}}$ 
4:   for  $i = 1$  to 4 do
5:      $y^{(i)} \leftarrow a_i^+(S)$ ; update  $S$ ; advance  $\Delta$ 
6:   if  $y^{(4)}.\text{STATUS} = \text{OPTIMAL}$  then
7:      $Y \leftarrow \text{success}$ 
8:   else if  $k \geq K$  then
9:      $Y \leftarrow \text{failure}$ 
10:  else
11:    // Inspection stage at  $q_{\text{DG}}$ : inspector commits  $\sigma = (\text{causal order}, \text{executed refutation})$ 
12:     $\hat{\ell} \leftarrow \text{PRIOR}(y^{(4)}.\text{STATUS})$   $\triangleright$  seed probe order
13:    repeat
14:       $(r, h) \leftarrow a_{\hat{\ell}}^-(S)$   $\triangleright$  inspectee best-responds: comply-repair or deflect
15:       $\tau^* \leftarrow \arg \min_{\tau \in \mathcal{T}(r, h)} \text{cost}(\tau)$  s.t. decisive
16:       $(\beta, \hat{\ell}^*) \leftarrow a_5.\text{VERIFY}(r, h, \tau^*, S)$   $\triangleright$  executed refutation (re-solve)
17:       $k \leftarrow k + 1$ 
18:      if  $\beta = 0$  then  $\triangleright$  layer
19:         $\text{confirmedcommit } \tilde{y} \in h$ ; resume  $\Delta$  downstream
20:         $\hat{\ell} \leftarrow \text{NEXTCAUSALLAYER}(\hat{\ell})$   $\triangleright$  layer cleared: descend causal order
21:    until  $\beta = 0 \vee k \geq K$ 
22:  return  $Y$ 

```

Relation to the baseline. The forward chain, the blackboard, and the retry budget are inherited unchanged. The replacement is localized: the single-judge diagnosis call and its confidence-routed return are substituted by the committed inspection stage, and the return edge of δ^- is driven by the executed verdict β rather than by a confidence. This locality is deliberate — it isolates the contribution (the inspection game) from the scaffolding (the FSM) and makes the commitment-order ablation, which randomizes the order in which layers are probed, the decisive experiment: if a permuted order performs as well as causal order, the thesis is falsified.

5. Experiments

The campaign tests whether a committed inspection policy improves *verifiable* root-cause attribution under layered faults—and whether it can match a strong single-judge LLM on naming accuracy—rather than whether any protocol can eventually repair a failed solve.

Claim and mechanism under test. We do not claim that Stackelberg inspection is uniquely able to name ℓ^* from a stack trace: at failure time every method sees the same blackboard, so a capable single-judge LLM may match first-shot accuracy. The intended footing is the conjunction of (i) accuracy that is at least competitive with single-judge diagnosis, (ii) *verifiable* attribution via commitment plus executed refutation, and (iii) cost/audit properties under the same budget K . Achieving (i) requires an *evidence-informed* commitment: the leader forms a *layer ranking* from the stack and agent histories (ranks or votes—not LLM-reported probabilities, which are poorly calibrated), commits ω , then probes with executed refutation. Status-prior-only ω is treated as a preliminary apparatus, not the final proposed policy.

Pre-registered expectations. Each experiment below states an *expected conclusion* fixed before the corresponding redesigned runs. The campaign is organized as three blocks: commitment-order ablation, a unified baseline comparison (internal and external protocols), and mechanism diagnostics. Tables marked *measured* report the evidence-informed redesign ($\omega_{\text{source}} = \text{evidence_rank}$, hybrid rank under the force-zero plant); tables marked *preliminary* are from the earlier status-prior ω implementation and do not substitute for the redesign. External Debate/Reflexion and Exp-C remain unrun at the time of writing.

5.1. Setup

Data. All methods use the demand-uncertainty TSLP ECR model of Section 3 on the exported dataset `prob_tslp_ecr_demand`. We start from the smoke instance `smoke_H4_Omega5` and extend to a small family of horizon/scenario configurations from the same generator. The forward pipeline, blackboard schema, strict-success rule (Table 1), and sandbox solver path are shared. An end-to-end smoke run on `smoke_H4_Omega5` with progressive knowledge injection terminated in *Practical Optimal* ($\hat{z} \approx z^* \approx 1.390 \times 10^6$, relative gap below the 10^{-2} threshold): the solver’s OPTIMAL status was therefore *not* a Spurious Optimal under the taxonomy above.

Model and budget K . Provider and model are held fixed within each comparison block and reported explicitly. Every method receives the same retry budget K (`max_retries`).

Injection protocol. Ground-truth root causes $a^* \in \{a_1, a_2, a_3\}$ are obtained by controlled injection: a defect is planted at an upstream layer (DataEngineer or ModelExpert) so that the surface symptom often appears downstream (e.g., a traceback or crash locus in generated code). This matches the motivation in Section 4.4 that failure evidence records a symptom locus, not a certified identity of ℓ^* . The label a^* is used only for evaluation, never in agent prompts.

Fairness. At failure time every method reads the same blackboard (solver status, stack trace, agent input–output history). Repair uses the same agent `backward_step` maps and the same re-solve verification path; methods differ only in how they select or order the implicated layer. No method receives privileged tools beyond its diagnosis policy.

5.2. Methods compared

- **(A) Proposed / ablation family.** `diagnosis_mode=stackelberg` with committed policy $\sigma = (\omega, \nu)$ and ν fixed as executed refutation. The leader forms an evidence-informed layer ranking (no confidence weights), then commits a probing order $\omega \in \{\text{causal}, \text{reverse}, \text{random}\}$ obtained by aligning that ranking with (or permuting against) the `data>model>code` skeleton.
- **(B) Internal baselines** (Exp-B, same table as C). (i) *adversarial*: single-judge LLM accusation plus accused-agent self-check; (ii) *sequential*: fixed reverse-order backward chain without an inspection commitment.
- **(C) External verification baselines** (Exp-B). (i) *Debate* (Du et al., 2023): multiple diagnosis personas argue for a fixed number of rounds; terminal positions are aggregated by majority vote (no LLM-reported confidence weights; ties broken by the status prior), then the accused layer enters the shared repair/re-solve path. (ii) *Reflexion* (Shinn et al., 2023): single-judge-style accusation augmented with verbal reflections after failed diagnose–repair attempts, conditioned upon in later diagnosis rounds, again sharing the same repair/re-solve

machinery. Internal and external protocols answer one baseline question and are reported together in Section 5.6; execution may stage (B) before (C) for cost control.

5.3. Metrics

The primary metric is *verified* root-cause attribution accuracy $\mathbf{1}\{\hat{a} = a^*\}$ where $\hat{a}$ is the layer supported by executed refutation (not merely last-comply credit), optionally stratified by injected origin and solver-status surface. For commitment-order contrasts we also report first-probe hit $\mathbf{1}\{\omega_1 = a^*\}$. Secondary metrics are SSR@K, probe rounds, and tokens. SSR counts only *Practical Optimal* outcomes in Table 1; a bare OPTIMAL that fails the z^* gap test (*Spurious Optimal*) does not contribute. For Exp. 5.7 we additionally record deflect rates, the fraction of compliances and deflections overturned by executed refutation, and analysis-mode episode payoffs (u_L, u_F) from Section 4.4.2. Highest SSR alone is not a sufficient claim: under strong models many protocols may eventually repair; the scientific contrast is verifiable attribution under commitment at competitive naming accuracy.

5.4. Result interpretation and provenance

Reading results against expectations. Pre-registered expectations are the reading grid for every table that follows: an outcome *supports*, *does not support*, or *falsifies* the registered expectation, and an unsupported expectation revises the design before further experimental spend rather than accumulating post-hoc explanations. The campaign has already exercised this rule once. The status-prior implementation failed its attribution expectations outright—last-comply attribution was 0/5 for every method in the baseline comparison (Table 6), and no order exceeded 1/5 in the ablation (Table 4)—and the recorded response was a redesign of the commitment apparatus (evidence-informed ranking plus verified attribution accounting), not a re-labelling of the old runs; the status-prior tables are retained below only as preliminary context. Under thin replication ($n=3$ per cell) we restrict ourselves to *directional* language—consistent sign and ordering patterns across cells—and reserve confirmatory claims for the expanded grid; SSR is never interpreted in isolation from attribution (Section 5.3).

Provenance and implementation versions. Every measured cell is produced by a single frozen implementation. Each run writes a machine-readable manifest (configuration, provider and model, terminal metrics) and a per-node event log (tokens, durations, comply–deflect responses); tables aggregate these records, and the manifests and event logs—not any intermediate summary file—are the authority for every number reported. Implementation versions are declared rather than silently mixed: the measured hybrid tables below predate a later hardening of the data layer, a deterministic catalog and validator constraining the data-access guide used in forward and backward passes. Because that hardening touches the same paths the diagnosis loop exercises, post-hardening replications are not assumed poolable with the current cells: the expanded grid re-establishes its tables under the hardened implementation, preceded by a no-plant end-to-end gate confirming that the un-faulted pipeline still terminates in *Practical Optimal*; each table carries the campaign, and hence the implementation freeze, it was produced under.

5.5. Exp-A: Commitment-order ablation

Question. Does the inspector’s committed probing order carry information under layered faults? Equivalently: holding v and K fixed, is causal-order commitment better than reverse or random commitment on first-probe hit and verified attribution?

Design. All cells use `diagnosis_mode=stackelberg` with evidence-informed ranking at commit time. We vary only $\omega \in \{\text{causal}, \text{reverse}, \text{random}\}$ on the injection grid of Section 5.1. Causal ω rotates the `data>model>code` skeleton so that the top-ranked layer is probed first; reverse and random break that alignment (and, per the redesign spec, do not reuse the rank tip), isolating order from naming skill.

Expected conclusion. Causal ω yields strictly higher first-probe hit and verified attribution than reverse, and at least weakly higher than random, at comparable token cost. If causal $\approx$ random/reverse, Proposition 1 is not supported and the ranking/alignment design is revised before baselines.

Measured results (evidence-informed; thin $n=3$). Table 3 reports the redesigned ablation on `smoke_H4_0omega5` with plant `me_force_zero_sea`, $K=3$, provider DashScope / `deepseek-v4-flash`, and `rank_method=hybrid` (force-zero smell tips `model_expert` without inventing a second diagnosis mechanism). Causal commitment achieves first-probe 3/3 and the only non-zero verified attribution (1/3); reverse and random are 0/3 on both primary metrics.

Table 3

Exp-A commitment-order ablation — **measured** (evidence-informed; $n=3$ /order; $K=3$; hybrid rank). Primary: first-probe hit and verified attribution.

ω	First-probe hit	Verified attr.	SSR@ K	Mean tokens
causal	3/3 (1.00)	1/3 (0.33)	2/3	1.08×10^5
random	0/3 (0.00)	0/3 (0.00)	0/3	1.26×10^5
reverse	0/3 (0.00)	0/3 (0.00)	0/3	1.15×10^5

Table 4

Exp-A preliminary (status-prior ω ; real $n=5$; last-comply attr.).

ω	First-probe hit	Attr. hit	SSR@ K	Mean tokens
causal	4/5 (0.80)	0/5	1/5	1.38×10^5
random	3/5 (0.60)	0/5	0/5	1.38×10^5
reverse	0/5 (0.00)	1/5	1/5	1.40×10^5

SSR@ K likewise concentrates on causal (2/3). These counts are directional under thin n ; they are not a definitive Proposition 1 proof.

Preliminary results (status-prior ω ; real $n=5$). Table 4 records the earlier status-prior run (last-comply attribution). The first-probe ordering is already directional (causal 4/5, random 3/5, reverse 0/5), but attribution collapses across all orders (at most 1/5): probing a layer and observing its backward response does not by itself establish guilt, including for the order that named the guilty layer first most often. Two limits of that apparatus follow. First, last-comply credit is not an attribution mechanism—the single reverse-order hit is last-comply credit, not a verified identification. Second, the status prior reads the failure surface, and surfaces mark symptom loci: on crash surfaces it points at the code layer regardless of where the defect was planted. Both limits motivated the evidence-informed redesign; the table is superseded as the primary Exp-A result by Table 3.

5.6. Exp-B: Baselines (internal and external)

Question. Relative to alternative diagnosis protocols that share the same blackboard, repair maps, and budget K , does evidence-informed causal Stackelberg achieve verified attribution that is at least competitive with a strong single-judge LLM, while retaining an auditable commit–probe–refute trail? Do stronger multi-call protocols (Debate, Reflexion) erase that advantage on accuracy, cost, or both?

Design. We compare causal Stackelberg (ω from evidence-informed ranking aligned to the causal skeleton) against the methods in Section 5.2: *internal* baselines (adversarial, sequential) and *external* verification baselines (Debate, Reflexion). Internal protocols stress fairness inside the same stack; external protocols stress the strong-LLM ceiling. Implementation may stage internal cells before external ones for cost control; scientifically they answer one comparison question and are reported in one table.

Expected conclusion. Verified attribution: Stackelberg $\geq$ adversarial $\gg$ sequential. SSR may favour sequential (blind repair). Debate may match Stackelberg on verified attribution but at higher tokens and lower audit coverage; Reflexion should not dominate both accuracy and cost. If adversarial or Debate strictly dominates Stackelberg on verified attribution at similar or lower cost with no audit gap worth claiming, the ranking/commitment design (or the paper claim) is revised before diagnostics scaling.

Measured results (evidence-informed; internal only; thin $n=3$). Table 5 reports the redesigned internal comparison on the same plant, model, and K as Exp-A, using the Exp-A causal cells as the Stackelberg arm. Verified attribution is 1/3 for causal Stackelberg and 0/3 for both adversarial and sequential (gate: SB $\geq$ adversarial). First-probe remains 3/3 under Stackelberg versus 1/3 adversarial. SSR@ K is highest for Stackelberg (2/3) on this draw; sequential’s mean token count is depressed by one zero-token failed episode (eb_seq_r1) and should be read with that caveat. Debate and Reflexion were not run in this pass (cost freeze until SB verified rates stabilize).

Table 5

Exp-B internal baselines — **measured** (evidence-informed; $n=3$; $K=3$). Primary: verified attribution. External Debate/Reflexion not run.

Family	Method	Verified attr.	First-probe	SSR@K	Mean tokens
Proposed	stackelberg (causal)	1/3 (0.33)	3/3 (1.00)	2/3	1.08×10^5
Internal	adversarial	0/3 (0.00)	1/3 (0.33)	1/3	1.84×10^5
Internal	sequential	0/3 (0.00)	0/2 (0.00)	1/3	0.61×10^5
External	Debate / Reflexion	—	—	—	not run

Table 6

Exp-B preliminary internal baselines (status-prior; real $n=5$).

Method	Attr. hit	First-probe hit	SSR@K	Mean tokens
stackelberg (causal)	0/5 (0.00)	4/5 (0.80)	1/5	1.38×10^5
adversarial	0/5 (0.00)	2/5 (0.40)	0/5	2.35×10^5
sequential	0/5 (0.00)	0/4 (0.00)	2/5	0.86×10^5

Table 7

Exp-C diagnostics — **illustrative mock** (verified attr. vs. K ; deflect overturn under causal Stackelberg).

Method / K	$K=1$	$K=2$	$K=3$	Deflect overturn
stackelberg causal	0.45	0.60	0.70	0.75
adversarial	0.50	0.55	0.60	—
sequential	0.10	0.15	0.20	—

Preliminary results (status-prior; real $n=5$, internal only). Table 6: last-comply attribution was 0/5 for all three methods, while SSR diverged (sequential repaired 2/5 blind, stackelberg 1/5, adversarial 0/5). The attribution failure is uniform across protocol families—single-judge accusation, committed causal probing, blind reverse-chain repair—so no protocol’s comply/deflect responses identified the planted layer under last-comply accounting; conversely, sequential’s blind-repair SSR shows that eventual repair is likewise uninformative about root cause. These two observations motivated the verified attribution accounting and the evidence-informed rank of the redesign. External baselines were not run in that preliminary pass.

5.7. Exp-C: Inspection diagnostics

Question. Is commitment operational in the implementation—not merely additional LLM calls? As the budget K grows, does verified attribution rise under causal commitment, and are inconsistent deflects overturned by executed refutation?

Design. We sweep K for causal Stackelberg (and, as references, adversarial and sequential) on the same injection grid, recording verified attribution versus K , deflect rates, and the fraction of deflects/compliances overturned by re-solve. Analysis-mode payoffs (u_L, u_F) from Section 4.4.2 may be reported as secondary trajectory summaries.

Expected conclusion. Verified attribution rises with K faster under causal Stackelberg than under random/reverse or non-committed baselines; deflects that contradict re-solve are overturned at high rate. Flat curves in K or near-zero overturn rates indicate that commitment is not operational.

Illustrative expected pattern (mock; not measured). Table 7.

5.8. Failure-mode analysis

Table-level rates do not reveal where the mechanism breaks, so we additionally inspected the per-node event logs and repair artifacts of every measured cell above and of the intervening single-run smokes. Three recurrent failure

families emerged; each is a property of the implemented pipeline under layered faults, and each motivated a specific element of the measured design or of the pending expanded grid.

Traceback-as-oracle. On crash surfaces the failure evidence points at the generated code, and ranking that trusts the surface tips the code layer. Under an intermediate evidence-informed variant with LLM-only ranking (same grid, $n=3$), two of three causal cells committed a code-first ω , spent the budget on code- and data-layer overturns, and never reached the model layer; verified attribution was 0/9 over that grid. This is the practical form of the symptom-locus ambiguity of Section 4.4.1: the traceback marks where the pipeline broke, not where the defect was planted. The hybrid ranking of the measured tables counters it with an explicit symptom-signature rule—a forced-zero balance is model-layer evidence—while still ignoring LLM-reported confidence.

Stale evidence at repair time. A correct tip does not guarantee a verified attribution: in that same intermediate variant, the one cell that tipped the model layer still failed, because its backward step consumed the pre-fault forward output, in which the planted constraint was absent; the repair rewrote an unrelated model element and was overturned by the re-solve. The measured implementation presents the repairing agent with the current (post-fault) output and restricts the repair to a minimal strip of the implicated constraint rather than a free regeneration.

Residual repair-path variance. The measured campaigns retain a regeneration instability downstream of a correct model-layer repair: a code-layer regeneration can fail outright (in a single-run smoke it raised a key error and left a large objective gap despite a correct strip); one sequential grid cell terminated before any model call; and replicate variance remains high—the adversarial arm reached a verified attribution in a single-run smoke but 0/3 on the grid. Success can also occur without attribution: one causal cell reached *Practical Optimal* through a downstream repair while its attributed layer settled on the code layer. These residuals concentrate in the paths the deterministic data-layer contract now constrains; the no-plant gate and the re-established tables of Section 5.4 bound them before the expanded grid is interpreted, and the overturn and deflect diagnostics of Exp. 5.7 will quantify them at scale.

Claim boundary. Comparisons are scoped to layered-error natural-language optimization on the TSLP ECR pipeline. We do not claim gains for general LLM reasoning, nor that inspection is the unique protocol that can eventually repair a failed solve. The intended positive claim is that evidence-informed causal-order committed inspection matches (or exceeds) strong single-judge naming accuracy while improving verifiable attribution and auditability relative to unordered commitment and to non-committed verification baselines under the injection regime above.

6. Conclusion

[Placeholder] To follow once Exp-A–C report attribution, cost, and mechanism diagnostics.

References

- Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.
- Y. Xiao et al. Chain-of-experts: When LLMs meet complex operations research problems. In *ICLR*, 2024. <https://openreview.net/forum?id=HobyL1B9CZ>.
- A. AhmadiTeshnizi, W. Gao, M. Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *ICML*, PMLR 235:577–596, 2024. <https://arxiv.org/abs/2402.10172>.
- A. AhmadiTeshnizi, W. Gao, H. Brunborg, S. Talei, M. Udell. OptiMUS-0.3: Using large language models to model and solve optimization problems at scale. *arXiv:2407.19633*, 2024.
- B. Yang, Q. Zhou, J. Li, C. Su, P. Angeloudis, S. Hu. ORThought: Benchmarking and automating logistics optimization modeling via structured LLM reasoning. *arXiv:2508.14410*, 2025.
- Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv:2305.14325*, 2023.
- T. Liang, Z. He, W. Jiao, et al. Encouraging divergent thinking in large language models through multi-agent debate. In *EMNLP*, pages 17889–17904, 2024.
- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- S. Zhang, M. Yin, J. Zhang, et al. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In *ICML*, PMLR 267:76583–76599, 2025. <https://arxiv.org/abs/2505.00212>.
- M. Chen, J. Wang, F. Mu, et al. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In *ACL*, 2026. <https://arxiv.org/abs/2604.22708>.

- Y. Wang, W. Wu, J. Wang, Q. Wang. From flat logs to causal graphs: Hierarchical failure attribution for LLM-based multi-agent systems. *arXiv:2602.23701*, 2026.
- J. Shah. Causal Agent Replay: Counterfactual attribution for LLM-agent failures. *arXiv:2606.08275*, 2026.
- From debate to equilibrium: Belief-driven multi-agent LLM reasoning via Bayesian Nash equilibrium. In *ICML*, 2025. <https://proceedings.mlr.press/v267/yi25c.html>.
- B. Wang. A Stackelberg framework for resource-aware LLM agents: Learning, repair, and conditional guarantees. *arXiv:2606.23026*, 2026.
- H. von Stackelberg. *The Theory of the Market Economy*. Oxford University Press, 1952.
- R. Avenhaus, B. von Stengel, S. Zamir. Inspection games. In R. J. Aumann and S. Hart (eds.), *Handbook of Game Theory with Economic Applications*, vol. 3, ch. 51, pp. 1947–1987. Elsevier, 2002.
- M. Maschler. A price leadership method for solving the inspector’s non-constant-sum game. *Naval Research Logistics Quarterly*, 13(1):11–33, 1966.
- M. L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *ICML*, 1994.
- R. Poiani, C. Stirbu, A. M. Metelli, M. Restelli. Optimizing empty container repositioning and fleet deployment via configurable Semi-POMDPs. *IEEE Transactions on Intelligent Transportation Systems*, 25(5):4704–4711, 2024. <https://doi.org/10.1109/TITS.2023.3329677>.
- W. Wang, C. Diao, W. He, Z. Jin, Z. Yang. Collaborative optimization of container liner slot allocation and empty container repositioning within port clusters. *Journal of Marine Science and Engineering*, 13(1):159, 2025. <https://doi.org/10.3390/jmse13010159>.
- J. Cai, Y. Huang, C. Diao, Z. Jin. Joint optimization of multi-period empty container repositioning and inventory control based on adaptive particle swarm algorithm. *Journal of Marine Science and Engineering*, 13(6):1113, 2025. <https://doi.org/10.3390/jmse13061113>.
- T. Liu, J. Yang, Y. Yin. Toward LLM-agent-based modeling of transportation systems: A conceptual framework. *Artificial Intelligence for Transportation*, 1:100001, 2025. <https://doi.org/10.1016/j.ait.2025.100001>.